数値計算の基礎

1．目的

数値計算には計算誤差の問題など、有限の値しか扱えないコンピュータ特有の問題を理解しておく必要があります。ここでは、「丸め誤差」、「桁落ち」、「情報落ち」などの誤差がどのようなものか実例を通して理解し、それらの影響をできるだけ小さくするプログラムの考え方を学びます。

2．技術的知識

（1）データ型の最大値と最小値

コンピュータでは、整数は32ビットという具合に、データは有限のビット列で表現されます。したがって、各データ型にはそのデータ型で表現できる最大値と最小値が決まっています。

各データ型が表現できる値の範囲は処理系に依存しており、標準ヘッダファイルの中で定義されています。整数（文字も含む）に関するデータ型については<limits.h>、実数に関するデータ型については<float.h>がそれにあたります。主なものを以下に示します。

<limits.h>

CHAR_MAX	：	char型で表すことのできる最大値
CHAR_MIN	：	char型で表すことのできる最小値
INT_MAX	：	int型で表すことのできる最大値
INT_MIN	：	int型で表すことのできる最小値
LONG_MAX	：	long型で表すことのできる最大値
LONG_MIN	：	long型で表すことのできる最小値
LLONG_MAX	：	long long型で表すことのできる最大値
LLONG_MIN	：	long long型で表すことのできる最小値

<float.h>

FLT_MAX	：	float型で表すことのできる最大値
FLT_MIN	：	float型で表すことのできる最小値
FLT_DIG	：	float型の１０進数で表すことのできる精度の桁数
DBL_MAX	：	double型で表すことのできる最大値
DBL_MIN	：	double型で表すことのできる最小値
DBL_DIG	：	double型の１０進数で表すことのできる精度の桁数
LDBL_MAX	：	long double型で表すことのできる最大値
LDBL_MIN	：	long double型で表すことのできる最小値
LDBL_DIG	：	long double型の１０進数で表すことのできる精度の桁数

あるデータ型の変数に、絶対値がその型で表すことのできる最大値を超えた値を代入したことにより、意味のない値になってしまうことを**オーバーフロー**といいます。また、実数型では、絶対値がそのデータ型で許される最小の値より小さい値を代入し、０（**ゼロ**）になってしまうことを**アンダーフロー**といいます。

（2）計算誤差

コンピュータを用いて実数（浮動小数点数）の計算をするときは、計算誤差を避けて通ることはできません。これは、コンピュータ上で表現される実数は数学で扱う実数と異なり、float型でもdouble型でも有限の桁しか持っていないからです（数学では桁数は無限）。そのため、実数を扱う計算では必ず誤差が発生し、場合によってはそれを無視することができません。

このような計算誤差には、**丸め誤差**，**桁落ち**，**情報落ち**があります。

・丸め誤差

10進数で表わすと有限桁の実数値でも、2進数にすると有限の桁数で表すことができないことがあります。例えば、10進数の0．1は2進数で表すと、

$$0.1_{(10)} = 0.0001100110011001100110011\cdots_{(2)}$$

のように循環小数になってしまい、有限桁で表すことができません。この場合、コンピュータ内部では、強制的にある桁数で打ち切って表すことになります。このことによって生じる誤差を**丸め誤差**(round-off error)といいます。

［丸め誤差が生じる例］ /home/pub/practice/round-off.c

```
#include <stdio.h>

main()
{
      int i;
      float s;

      s = 0;
      for (i = 0; i < 10000; i++) {
            s = s + 0.1;
      }
      printf("0.1を10000回加えると、%gになる。¥n", s);

      return(0);
}
```

丸め誤差は、次のような**積み残し処理**を行うことで補正することができます。

[誤差の補正（積み残し処理）] /home/pub/practice/round-off-cancel.c

```
#include <stdio.h>

main()
{
    int i;
    float s, work, loss, d;

    s = loss = 0;
    for (i = 0; i < 10000; i++) {
        work = s;
        d = 0.1 + loss;
        s = s + d;
        loss = d - (s - work);
    }
    printf("0.1を10000回加えると、%gになる。¥n", s);

    return(0);
}
```

このように、実数を扱う計算では丸め誤差が発生するため、実数どうしの比較を行う場合は注意が必要です。

また、数学的には正しい計算であっても、**実数を含む式を等号(==)で直接比較することは絶対避ける**必要があります。

[悪い例] /home/pub/practice/float-cmp1.c

```
#include <stdio.h>

main()
{
    double a;
    double b;

    a = 0.6 - 0.4;
    b = 0.2;
    if (a == b) {
        printf("(0.6 - 0.4) と 0.2 は等しい。¥n");
    } else {
        printf("(0.6 - 0.4) と 0.2 は等しくない。¥n");
    }

    return(0);
}
```

・**計算機 ε** (machine epsilon)

実数を含む式を等号で比較したい場合は、次のようにゼロに近い値 ε を仮定し、比較する数値の差の絶対値が、ε より小さいかどうかで判断する必要があります。この ε のことを**計算機 ε** (machine epsilon)と呼び、

1.0 + ε > 1.0 となる最小の値

として定義されます。また、ε の処理系固有の値は、データ型ごとに<float.h>で

float型	:	FLT_EPSILON
double型	:	DBL_EPSILON
long double型	:	LDBL_EPSILON

として定義されています。

計算機 ε を用いて、先ほどの[**悪い例**]は次のように書き直すとよいでしょう。

[**良い例**] /home/pub/practice/float-cmp2.c

```
#include <stdio.h>
#include <math.h>
#include <float.h>

main()
{
    double a;
    double b;

    a = 0.6 - 0.4;
    b = 0.2;
    if (fabs(a - b) < DBL_EPSILON) {
        printf("(0.6 - 0.4) と 0.2 は等しい。¥n");
    } else {
        printf("(0.6 - 0.4) と 0.2 は等しくない。¥n");
    }

    return(0);
}
```

・桁落ち

実数値を扱う演算で、計算結果が0に極端に近くなる加減算を行ったとき、演算結果の有効数字の桁数が極端に少なくなることがあります。

この現象を**桁落ち**(cancelling in digits)といいます。次の計算を見てみましょう。

$$0.123456789-0.123456788=0.000000001=1.0\times10^{-9}$$

この計算では、減算結果を正規化することによって、有効数字の桁数が9桁から一気に1桁に減少してしまいます。実数の値は、コンピュータの内部では常に有効数字の桁数を一定として扱っています(これにより小数点の位置が変化するので浮動小数点数という)。このため、桁落ちが発生すると、不足した桁数が自動的に0で埋められてしまい、それまで小さかった誤差が相対的に大きくなってしまいます。さらに、桁落ちした数値に大きな数を掛けるなどの計算を行うと、発生した誤差が上の桁に上がってくることによって、計算結果を無意味にするほどの大きな誤差を含んだ値が返されることになります。桁落ち自体による問題はコンピュータとは無関係に発生しますが、コンピュータ上での桁落ちは、計算途中の値が分からないばかりか、結果の桁数が常に一定なのでその発生に気づきにくく、プログラミングには注意が必要です。

[桁落ちが生じる例]

二次方程式 $ax^2+bx+c=0\ \ (a\neq0)$ の2つの実数解は、

$$x_1=\frac{-b+\sqrt{b^2-4ac}}{2a}\quad,\qquad x_2=\frac{-b-\sqrt{b^2-4ac}}{2a}$$

ですが、$b>0$ のときは $-b+\sqrt{b^2-4ac}$, $b<0$ のときは $-b-\sqrt{b^2-4ac}$ の計算で桁落ちが発生します。例えば、

$$x^2\ +\ 96325x\ +\ 0.74125\ =\ 0$$

の厳密解は、

$$x_1=\ -7.6953023624\times10^{-6}\quad,\qquad x_2=\ -9.6324999992\times10^{4}$$

となります。しかし、$\sqrt{b^2-4ac}$ の値を計算すると

$$\begin{aligned}\sqrt{b^2-4ac}&=\sqrt{96325.0^2-4\times0.741250}\\&=\sqrt{9278505622.035}\\&=\ 96324.999999846\cdots\end{aligned}$$

であり、これを有効桁数が10進6桁であるfloat型で表すと$\sqrt{b^2-4ac}$ =96325.0になります。

そのため、$-b+\sqrt{b^2-4ac}$ の値は、

$$-b+\sqrt{b^2-4ac} = -96325.0 + 96325.0 = 0$$

で、何と有効桁数が0になってしまいます。

［解の公式をそのまま使ったプログラム］ /home/pub/practice/kainokousiki.c

```
#include <stdio.h>
#include <math.h>

int main()
{
    float a, b, c, d, x1, x2;

    a = 1.0;
    b = 96325.0;
    c = 0.74125;

    d = sqrt(b * b - 4 * a * c);

    x1 = (-b + d) / (2 * a);
    x2 = (-b - d) / (2 * a);

    printf("x1 = %g, x2 = %g¥n", x1, x2);

    return(0);
}
```

（実行結果） x1 = 0, x2 = -96325

桁落ちは、絶対値が非常に近い値の減算をしなければ起きません。そこで、この例では、解と係数の関係を用い、次のようにすれば回避することができます。

$b>0$ ですから、まず桁落ちの生じない $x_2=\dfrac{-b-\sqrt{b^2-4ac}}{2a}$ を求めます。

次に解と係数の関係より、

$$x_1+x_2=-\frac{b}{a} \cdots ① \quad , \quad x_1x_2=\frac{c}{a} \cdots ②$$

が成り立ちますから、x_1は、加減算を含まない②式により $x_1=\dfrac{c}{ax_2}$ として計算します。

［解と係数の関係を用いたプログラム］ /home/pub/practice/kaitokeisu.c

```
#include <stdio.h>
#include <math.h>

int main()
{
    float a, b, c, d, x1, x2;

    a = 1.0;
    b = 96325.0;
    c = 0.74125;

    d = sqrt(b * b - 4 * a * c);

    x2 = (-b - d) / (2 * a);
    x1 = c / (a * x2);

    printf("x1 = %g, x2 = %g¥n", x1, x2);

    return(0);
}
```

（実行結果） x1 = -7.6953e-06, x2 = -96325

有効桁数6桁に注目すれば、厳密解が得られていることがわかります。

・情報落ち（積み残し誤差）

大きさの著しく異なる2つの実数を加算した結果、小さいほうの有効桁が失われることがあります。この現象を**情報落ち**(loss of trailing digit)または**積み残し誤差**といいます。

例えば、

123456 + 12.3456 = 123468.3456

ですが、これを有効桁数6桁として計算すると

123456 + 12.3456 = 123468

になり、12.3456 の 0.3456 の部分が、計算結果に何の影響も及ぼさなくなっています。

［プログラム例］ /home/pub/practice/losstrail.c

```
#include <stdio.h>

main()
{
    float a, b, c;

    a = 123456;
    b = 12.3456;
    c = a + b;

    printf("%g + %g = %g¥n", a, b, c);

    return(0);
}
```

（実行結果）

```
123456 + 12.3456 = 123468
```

したがって、次のような級数の計算を行う場合も注意が必要になります。

$$s = \sum_{k=1}^{n} \frac{1}{k(k+1)}$$

この級数は数学的に解くと、

$$s = \sum_{k=1}^{n} \frac{1}{k(k+1)} = \sum_{k=1}^{n} \left(\frac{1}{k} - \frac{1}{k+1}\right) = 1 - \frac{1}{n+1}$$

ですから、$n=9999$　のときは $s=1-\frac{1}{10000}=0.9999$となります。これをプログラムで繰り返しを用いて計算する場合、次の2通りの方法が考えられます。

（大きい値から加える）$s = \frac{1}{1\cdot2}+\frac{1}{2\cdot3}+\frac{1}{3\cdot4}+ \cdots +\frac{1}{9998\cdot9999}+\frac{1}{9999\cdot10000}$

（小さい値から加える）$s = \frac{1}{9999\cdot10000}+\frac{1}{9998\cdot9999}+\frac{1}{9997\cdot9998}+ \cdots +\frac{1}{2\cdot3}+\frac{1}{1\cdot2}$

実際、この両方を次のプログラム例で実行してみましょう。

［級数のプログラム例］ /home/pub/practice/series.c

```
#include <stdio.h>

main()
{
     float sum;
     int i;

     sum = 0;
     for (i = 1; i <= 9999; i++) {
          sum = sum + 1.0 / (i * (i + 1.0));
     }
     printf("大きい方からの和 = %g¥n", sum);

     sum = 0;
     for (i = 9999; i >= 1; i--) {
          sum = sum + 1.0 / (i * (i + 1.0));
     }
     printf("小さい方からの和 = %g¥n", sum);

     return(0);
}
```

（実行結果）

```
大きい方からの和 = 0.999853
小さい方からの和 = 0.9999
```

実行結果を見ると、小さい値から加えた結果が数学的に計算した結果と一致していることがわかります。これは、大きい値から加えていくと、情報落ちの影響が大きくなるからです。したがって、実数を数多く加える計算では、できるだけ小さい値から加えて行く必要があります。また、可能な限り精度のよいデータ型を用いるべきです。

３．課題Ａ

（１）現在使用している処理系で、次のことを調べるプログラムを作りなさい。

(a) char, int, long, long longの各型で表すことのできる、整数の最大値と最小値。

（添付ファイル：numeric1.c）

(b) float, double, long doubleの各型で表すことのできる、実数値の最大値，最小値，計算機 ε および、10進数で表現できる有効桁数。

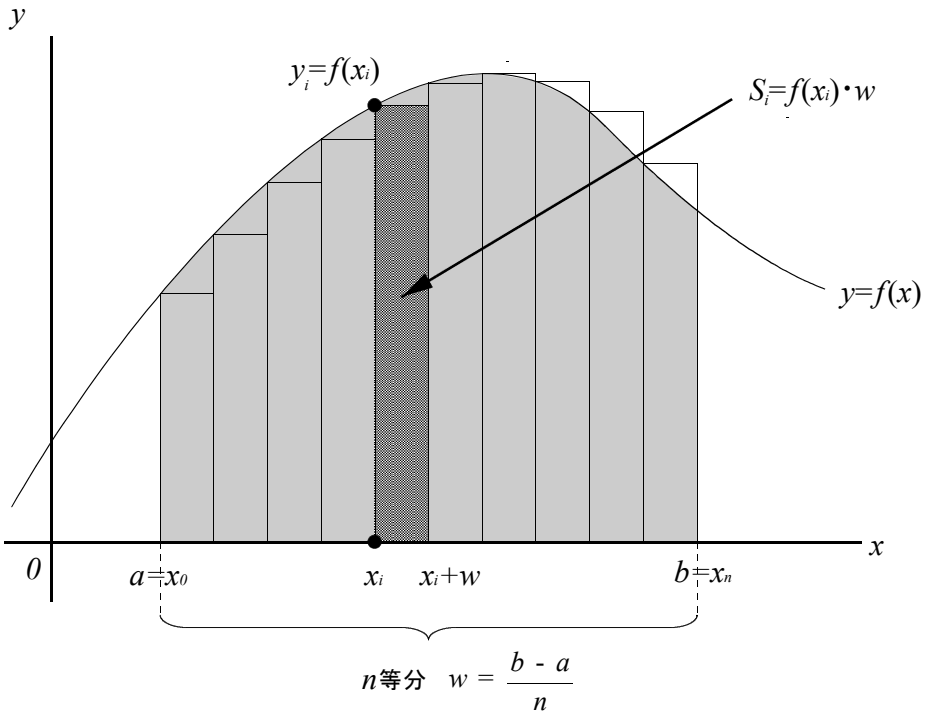
（添付ファイル：numeric2.c）

（２）区分求積法によりπを求める

コンピュータで定積分を求める最も単純な方法に、区分求積法というものがあります。これは、[図1]のように、区間[a, b]を n 等分し、各 $x_0, x_1, \cdots, x_i, \cdots, x_{n-1}$にできる、

高さ $f(x_i)$ と幅 $w=\dfrac{b-a}{n}$ の長方形の面積の総和 $S = \sum_{i=0}^{n-1} S_i = \sum_{i=0}^{n} f(x_i) \cdot w$ で近似

するもので、$n \to \infty$ の時の極限値は定積分 $\int_a^b f(x)dx$ に等しくなります。



[図1]区分求積法による定積分

円 $x^2+y^2=1$ の第1象限の面積が $\frac{\pi}{4}=\int_0^1 \sqrt{1-x^2}\,dx$ であることを利用して、区分求積法でπを小数点以下6桁まで正確に求めるプログラムを作りなさい。このとき、区間の分割数 n をどのくらいにするのがよいか検討しなさい。

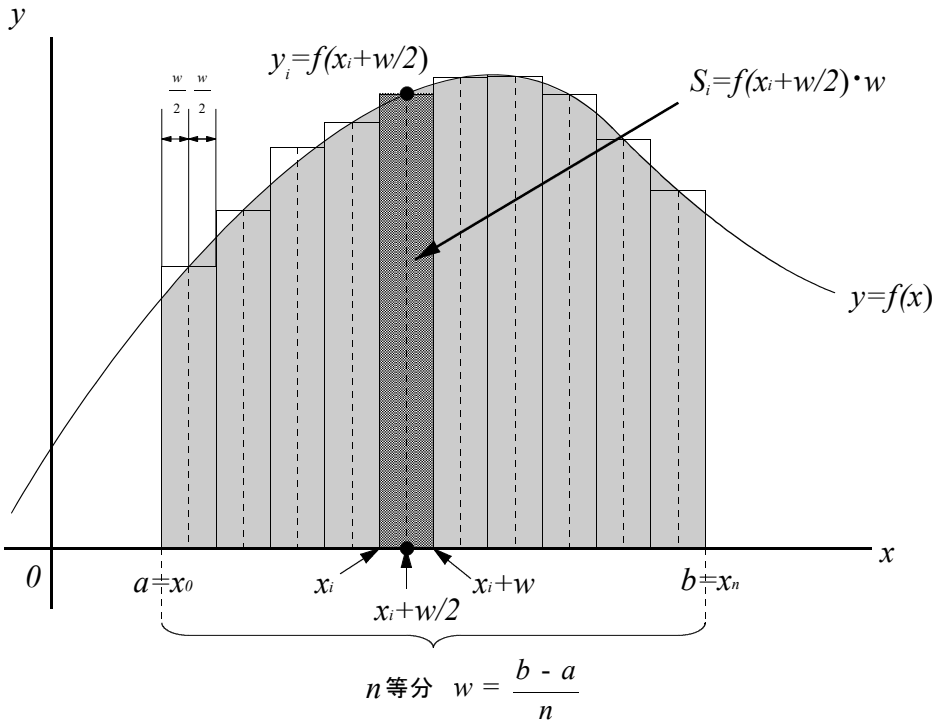
（添付ファイル：pi1.c）

（3）中点法によりπを求める

（2）の区分求積法において、区間[a, b]を n 等分して長方形を作る際、[図2]のように各長方形の高さを $f(x_i)$ ではなく、$f(x_i+\frac{w}{2})$ とする方法を中点法といいます。

この方法により、πを小数点以下6桁まで正確に求めるプログラムを作りなさい。また、区分求積法と中点法とで分割数 n を同じにしたとき、それぞれの精度を比較しなさい。

（添付ファイル：pi2.c）



[図2]中点法による定積分

（4）$tan^{-1}x$ の級数展開公式によりπを求める

$tan^{-1}x$ $(-1 \leqq x \leqq 1)$ をテイラー展開によって級数展開すると、

$$tan^{-1}x=\sum_{n=1}^{\infty}(-1)^{n-1}\frac{x^{2n-1}}{2n-1} = x-\frac{x^3}{3}+\frac{x^5}{5}-\frac{x^7}{7}+ \cdots + (-1)^{n-1}\frac{x^{2n-1}}{2n-1} + \cdots$$

となります（この式は、発見者の名前を取ってグレゴリー級数と呼ばれています）。

$tan\frac{\pi}{4}=1$ですから、$\frac{\pi}{4}=tan^{-1}1$より、π は、

$$\pi = 4tan^{-1}1 = 4\sum_{n=1}^{\infty}(-1)^{n-1}\frac{1}{2n-1} = 4\{1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+ \cdots + (-1)^{n-1}\frac{1}{2n-1} + \cdots \}$$

で計算できます。この式を用いて、π を小数点以下 6 桁まで正確に求めるプログラムを作りなさい。

なお、この式は無限級数となりますから、実際のプログラムでは、どの時点で計算を打ち切ればよいか考える必要があります。

（添付ファイル：pi3.c）

（参考）打ち切り誤差と相対打ち切り誤差

k-1 項までの和を *d*，*k* 項までの和を *s* としたとき、$|s-d|$ を**打ち切り誤差**、$\frac{|s-d|}{|d|}$ を**相対打ち切り誤差**といいます。一般に、

$$\frac{|s-d|}{|d|} < \text{必要精度}$$

となったときに、計算を打ち切ればよいことになります。必要精度は、10 進 6 桁程度ならば1.0×10^{-6}とすればよいでしょう。

4．課題B

（1） 数値積分について次のことを調べ、区分求積法と比べ収束の速さや精度について比較しなさい。

また、余力のある者は、これらの方法でπを求めるプログラムを作り、区分求積法との違いを実際に確かめなさい。

- 台形法
- シンプソン法

（2）テイラーの定理および、以下の関数について級数展開式を調べなさい。

- $sin(x)$
- $cos(x)$
- e^x
- $log_e(1+x)$